

Università di Ferrara
Laurea Triennale in Informatica
A.A. 2025-2026
Sistemi Operativi e Laboratorio

**2. Introduzione a UNIX shell
e file comandi**

Prof. Carlo Giannelli

`https://unife.coursecatalogue.cineca.it/insegnamenti/2025/46693/2016/9999/10431`

`http://docente.unife.it/carlo.giannelli`

`https://ds.unife.it/team/carlo-giannelli`

Shell

Programma che permette di far **interagire l'utente (interfaccia testuale) con SO tramite comandi** □ resta in attesa di un comando...

- ... mandandolo in esecuzione alla pressione di <ENTER>

In realtà (lo vedremo ampiamente) **shell è un interprete comandi evoluto**

- potente **linguaggio di scripting**
- interpreta ed esegue comandi da **standard input** o da **file comandi**

Differenti shell

- La shell non è unica, un sistema può metterne a disposizione varie
 - ❑ **Bourne shell** (standard), C shell, Korn shell, ... ❑
L'implementazione della **bourne shell in Linux** è **bash (/bin/bash)**
- Ogni utente può indicare la shell preferita
 - ❑ La scelta viene memorizzata in **/etc/passwd**, un file contenente le informazioni di tutti gli utenti del sistema
- La shell di login è quella che richiede inizialmente i dati di accesso all'utente
 - ❑ Per **ogni utente connesso** viene generato un **processo dedicato** (che esegue la shell)

Ciclo di esecuzione della shell

```
loop forever
  <LOGIN>
  do
    <ricevi comando da file di
    input> <interpreta comando>
    <esegui comando>
    while (! <EOF>)
  <LOGOUT>
end loop
```

Accesso al sistema: login

Per accedere al sistema bisogna possedere una coppia **username e password**

❑ NOTA: UNIX è case-sensitive

SO verifica le credenziali dell'utente e manda in esecuzione la sua **shell di preferenza**, posizionandolo in un **direttorio di partenza**

❑ Entrambe le informazioni si trovano in **/etc/passwd**

Comando **passwd**

- È possibile **cambiare la propria password** di utente, mediante il comando **passwd**
- Se ci si dimentica della password, bisogna chiedere all'amministratore di sistema (utente root)

Uscita dal sistema: **logout**

- Per uscire da una shell qualsiasi si può utilizzare il comando **exit** (che invoca la system call **exit()** per quel processo)
- Per uscire dalla shell di login
 - **logout**
 - **CTRL+D** (che corrisponde al carattere <EOF>)
 - **CTRL+C**
- Per rientrare nel sistema bisogna effettuare un nuovo login

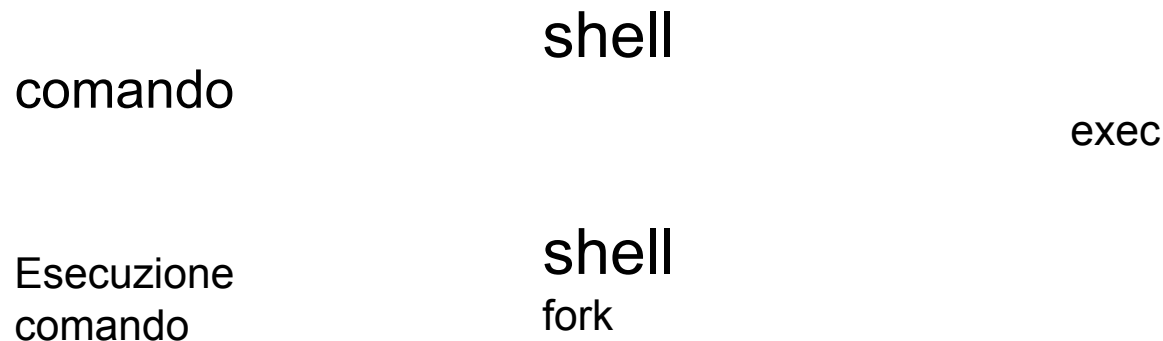
File Comandi in Shell 6

Esecuzione di un comando

- Ogni comando richiede al SO l'esecuzione di una

particolare azione

- I **comandi principali** del sistema si trovano nella directory **/bin**
- Possibilità di **realizzare nuovi comandi (scripting)** •
Per ogni comando, shell **genera un processo figlio dedicato alla sua esecuzione**
 - ❑ Il processo padre **attende la terminazione del comando** (foreground) o **prosegue in parallelo** (background)



Comandi e input/output

❑ **I comandi UNIX si comportano come FILTRI** ❑ un filtro è un programma che riceve un ingresso da un input e produce il risultato su uno o più output

Standard input

Standard
error

FILTRO

Default: tastiera
Standard
output

Default: monitor

File Comandi in Shell 8

Manuale

Esiste un **manuale on-line** (**man**), consultabile per informazioni su ogni comando Linux. Indica:

- ❑ **formato del comando (input) e risultato atteso (output)**
- ❑ **descrizione delle opzioni**
- ❑ possibili restrizioni
- ❑ file di sistema interessati dal comando
- ❑ comandi correlati
- ❑ eventuali bug

Per uscire dal manuale, digitare :q (quit per editor tipo vi)

File Comandi in Shell 9

Formato dei comandi

Tipicamente: ***nome –opzioni argomenti***

Esempio: `ls -l temp.txt`

- Convenzione nella rappresentazione della sintassi comandi:
- se un'opzione o un argomento possono essere omessi, si indicano tra quadre **[opzione]**
 - se due opzioni/argomenti sono mutuamente esclusivi, vengono separati da **| arg1 | arg2**
 - quando un arg può essere ripetuto n volte, si aggiungono dei puntini **arg...**

File Comandi in Shell 10

Cenni pratici introduttivi all'utilizzo del filesystem Linux

File

File come **risorsa logica** costituita da **sequenza di bit**, a cui viene dato un nome

Astrazione molto potente che consente di **trattare allo stesso modo entità fisicamente diverse** come file di testo, dischi rigidi, stampanti, direttori, tastiera, video, ... •

Ordinari

- ❑ archivi di dati, comandi, programmi sorgente, eseguibili, ...

• **Directory**

- ❑ gestiti direttamente solo da SO, contengono riferimenti a file

• **Speciali**

- ❑ dispositivi hardware, memoria centrale, hard disk, ...

In aggiunta, anche:

- FIFO (pipe) - file per la comunicazione tra processi
- soft link - riferimenti (puntatori) ad altri file o direttori

File: nomi

- È possibile nominare un file con una **qualsiasi sequenza di caratteri (max 255)**, a eccezione di '.' e '..'
- È sconsigliabile utilizzare per il nome di file dei caratteri speciali, ad es. **metacaratteri e segni di punteggiatura** • Ad ogni file possono essere associati **uno o più nomi simbolici (link)** ma ad ogni file è associato **uno e un solo descrittore (i-node)** identificato da un intero (i-number)

Filesystem Linux è
organizzato come un grafo
diretto aciclico (DAG)

/

radice (root)

etc

users webeggs

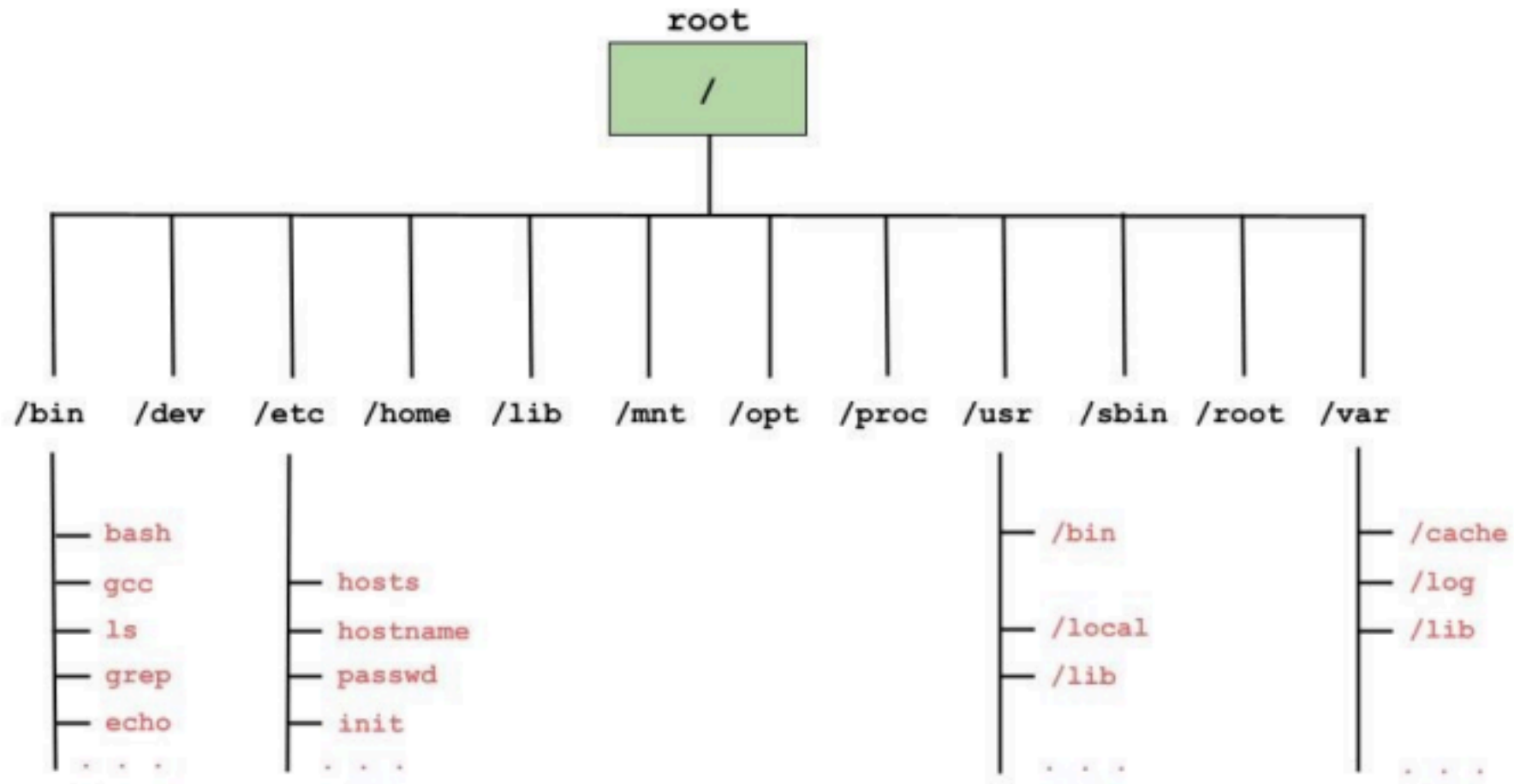
home bin dev

bash

README

info

link link



NB: rappresentazione **NON** universale, alcuni filesystem potrebbero contenere directory non presenti in altri

Filesystem Linux

- **bin:** directory contenente i binaries (applicazioni e programmi eseguibili)
- **dev:** file relativi ai device, generati a tempo di boot o anche “on the fly” (es. collegamento chiavetta USB)
- **etc:** file di configurazione di sistema
- **home:** qui si trovano le directory personali di ciascun utente
- **lib:** contiene le librerie con il codice disponibile alle applicazioni (es. aprire una finestra, salvare un file nell’hard disk)
- **proc:** contiene informazioni quali la CPU o il Kernel in cui il SO Linux sta eseguendo

Filesystem Linux

- **root:** directory home dell’amministratore del sistema (non confondere con /)

- **sbin**: simile a bin ma contenente solamente applicazioni eseguibili dall'amministratore (accessibili attraverso il comando sudo)
- **usr**: originariamente conteneva le home directory degli utenti. Oggi principalmente contiene le risorse che necessitano di essere condivise tra applicazioni e servizi
→ qual è la differenza tra bin e usr/bin?
- **var**: contiene variabili e altri dati che tendono a variare di frequente (es. var/log contiene i file di log, ovvero file in cui si tiene traccia degli eventi accaduti nel SO)

Gerarchie di directory

- All'atto del login, l'utente può cominciare a operare all'interno di una specifica directory (**home**). In

seguito è possibile cambiare directory

- È possibile visualizzare il percorso completo attraverso il **comando** `pwd` (print working directory)
- Essendo i file organizzati in **gerarchie di directory**, SO mette a disposizione dei comandi per muoversi all'interno di essi

File Comandi in Shell 18

Nomi relativi/assoluti

Ogni utente può specificare un file attraverso ☐ **nome relativo**: è riferito alla posizione dell'utente nel filesystem (direttorio corrente)

☐ **nome assoluto**: è riferito alla radice della

gerarchia /

Nomi particolari

- ❑ `.` è il direttorio corrente (visualizzato da `pwd`) ❑
- `..` è il direttorio 'padre'
- ❑ `~` è la propria home utente

Il comando **`cd`** **permette di spostarsi all'interno del filesystem**, utilizzando sia nomi relativi che assoluti ❑
`cd` senza parametri porta alla home dell'utente

Nomi relativi/assoluti:

esempi

o /

direttorio
corrente

etc

home bin dev

users local

README

nome assoluto: `/home/local/README`

nome relativo: `../local/README`

Gestione file: comando **ls**

Consente di **visualizzare nomi di file**

- varie opzioni: esempio **ls -l** per avere più informazioni (non solo il nome del file)
- possibilità di usare **metacaratteri (wildcard)**
 - ❑ Per es. se esistono i file **f1, f2, f3, f4**
 - ci si può riferire a essi scrivendo: **f***
 - o più precisamente **f[1-4]**

Più avanti studieremo meglio i metacaratteri e le modalità con cui vengono gestiti esattamente dalla shell

Opzioni del comando ls...

`ls [-opzioni...] [file...]`

Alcune opzioni

- ❑ **l** (long format): per ogni file una linea che contiene **diritti**, **numero di link**, **proprietario** del file, **gruppo** del proprietario, **occupazione di disco** (blocchi), **data e ora** dell'ultima modifica o dell'ultimo accesso e **nome**
- ❑ **t** (time): la lista è **ordinata per data** dell'ultima modifica
- ❑ **u** la lista è ordinata per data dell'ultimo accesso
- ❑ **r** (reverse order): inverte l'ordine
- ❑ **a** (all files): fornisce una **lista completa** (normalmente i file il cui nome comincia con il punto non vengono visualizzati)

- ❑ **F** (classify): indica anche il tipo di file (eseguibile: *, directory: /, link simbolico: @, FIFO: |, socket: =, niente per file regolari)

Link

Le informazioni contenute in uno **stesso file** possono essere **visibili come file diversi**, tramite “riferimenti” (link) allo stesso file fisico

- SO considera e gestisce la molteplicità possibile di riferimenti:
 - ❑ se un file viene cancellato, le **informazioni sono veramente eliminate solo se non ci sono altri link a esso**
 - ❑ Il link **cambia i diritti?** → meglio di no

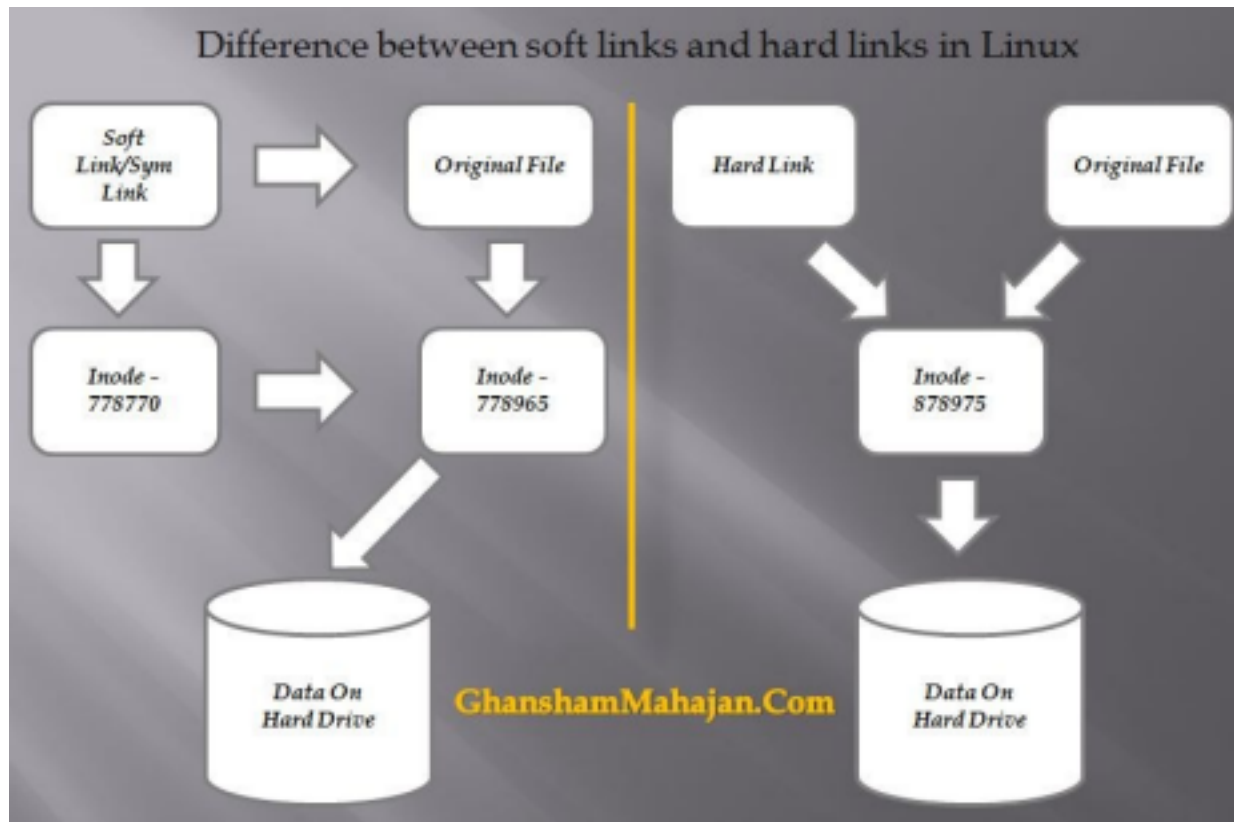
Due tipi di link:

- **link fisici** (si collegano le strutture del filesystem)

- **link simbolici** (si collegano solo i nomi)

Soft Link vs. Hard link

- Link fisici (hard link): l'hard link ha lo **stesso inode** del file originario
- Link simbolici (soft link o symlink): si collegano solo i nomi di file, soft-link e file originario hanno **inode distinti**



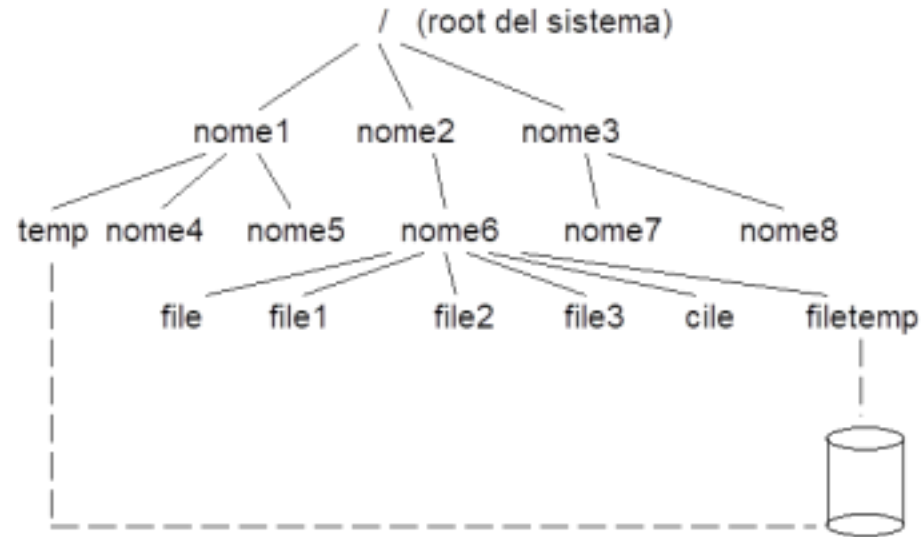
File Comandi in

Shell 24

Comando In, link

- Le informazioni contenute in uno stesso file possono essere visibili come file diversi, tramite “riferimenti” (link) allo stesso file fisico.

- SO considera e tratta il tutto: se un file viene cancellato, le informazioni sono veramente eliminate solo se non ci sono altri link a esso



comando: In [-s]

- In `/nome2/nome6/filetemp /nome1/temp`

File Comandi in Shell 25

Comandi vari di gestione

- Creazione/gestione di directory
 - ❑ **mkdir** <nomedir> *creazione di un nuovo direttorio*
 - ❑ **rmdir** <nomedir> *cancellazione di un direttorio* ❑

cd <nomedir> *cambio di direttorio*

❑ **pwd** *stampa il direttorio corrente*

❑ **ls** [<nomedir>] *visualizz. contenuto del direttorio*

• Trattamento file

❑ **ln** <vecchionome> <nuovonome> *link*

❑ **cp** <filesorgente> <filedestinazione> *copia* ❑ **mv** <vecchionome> <nuovonome> *rinom. / spost.* ❑ **rm** <nomefile> *cancellazione*

❑ **cat** <nomefile> *visualizzazione*

File Comandi in Shell 26

cat

Il comando cat stampa a video il contenuto di un file:

```
cgianelli@linuxdid:~$ cat /etc/passwd
```

```
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```
n bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
...
```

/etc/passwd is a text file that contains the attributes of (i.e., basic information about) each user or account on a computer running Linux or another Unix-like operating system

File Comandi in Shell 27

more (e less)

- Il comando **cat** effettua una semplicissima stampa del contenuto a video di un file. Per una visualizzazione a pagine, certamente più comoda per file di larghe dimensioni, è necessario utilizzare un comando di paginazione.

- Il comando **more** permette di visualizzare un file una pagina per volta.
- Esistono diverse alternative più recenti e potenti al comando **more**, tra cui va sicuramente segnalato il comando **less** (che permette anche di far scorrere la visualizzazione all'indietro).

File Comandi in Shell 28

echo

- Il comando **echo** stampa a video la stringa passatagli come parametro:

```
cgianelli@linuxdid:~$ echo Hello  
world! Hello world!
```

- Il comando **echo** rappresenta la primitiva fondamentale che la shell mette a disposizione per stampare informazioni a video.

File Comandi in Shell 29

sort

- Il comando **sort** riordina le righe di un file in ordine alfabetico
- Esempi e opzioni:

```
sort nomefile.txt # elenca le righe del file nomefile.txt  
                  # in ordine alfabetico crescente
```

```
sort -r nomefile.txt# elenca le righe del file nomefile.txt  
                  # in ordine alfabetico decrescente
```

- Il comando **sort** ha molte opzioni:
 - ❑ **-o <nomefileout>** stampa su file
 - ❑ **-n** interpreta le righe del file come numeri
 - ❑ **-k <n>** ordina il file secondo il contenuto della n-esima colonna

File Comandi in Shell 30

diff

- Il comando **diff** stampa la differenza tra il contenuto di due file
- Esempio:

```
diff <file1> <file2> # mostra (solo) le righe diverse
```
- Il comando **diff** è molto utilizzato nella gestione del codice sorgente

WC

- Il comando **wc** stampa il numero di righe, parole, o caratteri contenuti in un file

- Esempi e opzioni:

`wc -l nomefile.txt` # conta le **linee** (opzione l)

contenute nel file nomefile.txt

`wc -w nomefile.txt` # conta le **parole** (opzione w) #

contenute nel file nomefile.txt

`wc -c nomefile.txt` # conta i **caratteri** (opzione c) #
contenuti nel file nomefile.txt

File Comandi in Shell 32

grep

- Il comando **grep** seleziona le righe di un file che contengono la stringa passata come parametro e le stampa a video

- Esempi:

`grep stringa nomefile.txt` # **seleziona** le righe del file #
nomefile.txt che contengono il testo “stringa”
e le stampa a video

grep -c stringa nomefile.txt # **conta** le righe del file #
nomefile.txt che contengono il testo “stringa”
e stampa il numero a video

File Comandi in Shell 33

grep

grep -r stringa dir # seleziona le righe di tutti i file #
nella directory dir che contengono il
testo “stringa” e le stampa a video,
ricorsivo sulle sotto directory

- Il comando grep ha numerosissime opzioni ed è utilizzatissimo, specialmente nella gestione e manipolazione di file di configurazione e di codice sorgente

head

- Il comando **head** mostra le prime righe di un file

- Esempi e opzioni:

`head -n 15 nomefile.txt` # mostra le prime 15 righe del file

`head -c 30 nomefile.txt` # mostra i primi 30 caratteri del file

tail

- Il comando **tail** mostra le ultime righe di un file

- Esempi e opzioni:

`tail -n 15 nomefile.txt # mostra le ultime 15 righe del file`
`tail -c 30 nomefile.txt # mostra gli ultimi 30 caratteri del file`

time

- Il comando **time** cronometra il tempo di esecuzione di un comando
- Esempi:
- `cgianelli@linuxdid:~$ time ls foto.jpeg`
`foto.jpeg`

ls foto.jpeg 0,00s user 0,00s system 0% cpu 0,002

total File Comandi in Shell 37

who

- Il comando **who** mostra gli utenti attualmente collegati al

sistema (ovverosia che hanno eseguito il login sulla macchina)

- Esempio:

```
cgiannelli@linuxdid:~$ who
```

```
cgiannelli :0 2013-12-15 11:01
```

```
cgiannelli pts/0 2013-12-15 11:02 (:0)
```

```
cgiannelli pts/1 2013-12-15 12:14 (:0)
```

```
cgiannelli pts/3 2013-12-16 18:19 (:0)
```

File Comandi in Shell 38

man

- In ambiente Unix, il comando **man** rappresenta l'help di sistema

- Esempio:

`man grep` # mostra l'help (guida) del comando `grep`

- Oltre al comando `man`, le moderne distribuzioni di Linux mettono a disposizione anche il comando **info**

File Comandi in Shell 39

ps

Un processo utente in genere viene attivato a partire da un comando (da cui prende il nome). Ad es., dopo aver mandato in esecuzione il comando `hw`, verrà visualizzato un processo dal nome `hw`.

Tramite **ps** si può vedere la lista dei processi attivi

```
cgianelli@info1-linux:~$ ps
```

PID	TTY	STAT	TIME	COMMAND
4837	p2	S	0:00	-bash
6945	p2	S	0:00	sleep 5s
6948	p2	R	0:00	ps

Comando **ps** molto utile quando si lancia l'esecuzione di programmi di sistema con errori di programmazione (guardare su man le varie opzioni **ps**)

File Comandi in Shell 40

ps

```
cgianelli@linuxdid:~$ ps au
```



```
user@YOGA-CG-UNIFE:~$ ps au
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         6  0.0  0.0   8900    216 tty1      Ss   11:40   0:00 /init ro
user        7  0.0  0.0  20076   6708 tty1      S   11:40   0:00 -bash
user       92  0.0  0.0  17380   1920 tty1      R   11:59   0:00 ps au
user@YOGA-CG-UNIFE:~$
```

Cosa è il PID?

File Comandi in Shell 41

ps

Alcune opzioni importanti:

- a Mostra anche i processi degli altri utenti
- u Fornisce nome dell'utente che ha lanciato il processo e l'ora in cui il processo è stato lanciato

- x Mostra anche i processi senza terminale di controllo (processi demoni)

Attenzione: si ricordi di non usare il trattino (-) per specificare le opzioni del comando ps, in quanto ps adotta la sintassi di tipo “extended BSD” che non prevede l’uso di trattini

Se si vuole un aggiornamento periodico dello stato dei processi correnti, si usi il comando **top**

File Comandi in Shell 42

top

- Il comando **top** fornisce una visione dinamica in real-time del sistema corrente. Esso visualizza continuamente informazioni sull’utilizzo del sistema (memoria fisica e virtuale, CPU, ecc.) e sui processi che usano la maggiore

share di CPU.

- Esiste anche una versione più avanzata e moderna di top, chiamata **htop**. Al contrario di top, di solito htop non è incluso tra le utility di base disponibili sulle macchine Linux e va esplicitamente installato.

File Comandi in Shell 43

pgrep e pkill

Il comando **pgrep** restituisce i pid (process id) dei processi che corrispondono alle caratteristiche richieste (nome processo, utente, ...).

Ad esempio, per ottenere il pid del processo Skype:

```
cgianelli@linuxdid:~$ pgrep skype
```

```
9280
```

Per ottenere il pid del processo demone sshd appartenente all'utente root: `cgiannelli@linuxdid:~$ pgrep -u root sshd`
3488

pkill presenta la stessa interfaccia comandi di `pgrep`, ma anziché stampare a video il pid dei processi, li uccide (in realtà invia loro un segnale SIGTERM).

File Comandi in Shell 44

Terminazione forzata di un processo

È possibile 'terminare forzatamente' un processo tramite il comando `kill`

Ad esempio:

- `kill -9 <PID>` provoca l'invio di un segnale **SIGKILL** (forza la terminazione del processo che lo riceve e non può essere ignorato) **al processo identificato da PID**

□ Esempio: `kill -9 6944`

Per conoscere il PID di un determinato processo, si può utilizzare il comando `ps` oppure il comando `pgrep`

File Comandi in Shell 45

Segnali e interruzioni

È possibile **interrompere un processo** (purché se ne abbiano i diritti...) `kill -s <PID>`

provoca l'**invio di un segnale** (individuato dal parametro `s`) al processo identificato dal PID - vedremo ampiamente il

concetto di segnale più avanti nel corso...

`kill -9` è solo un esempio: **9 corrisponde a SIGKILL**, che provoca la **terminazione incondizionata del processo (segnale non mascherabile) e dei figli (ricorsivamente)**

Alcuni tra i segnali più comuni:

- × `CTRL-C` (invia un `SIGINT`, terminazione del processo attualmente in foreground, `kill -2`)
- × `CTRL-Z` (invia un `SIGTSTP`, sospensione di un processo, `kill`

`-20) kill -l` fornisce la lista dei segnali

File Comandi in Shell 46

Utenti e gruppi

Utenti e gruppi

- **Sistema multiutente** ⇒ problemi di privacy e di possibili interferenze: necessità di

proteggere/nascondere informazione •

Concetto di gruppo (es. staff, utenti, studenti, ...):
possibilità di lavorare sugli stessi documenti • **Ogni utente appartiene a un gruppo** ma può far parte anche di altri a seconda delle esigenze e configurazioni

- Comandi relativi all'identità dell'utente

- ❑ **whoami**

- ❑ **id**

File Comandi in Shell 48

Informazioni legate ai file

-l total 8

tot. spazio occupato (blocchi)

```
host133-63:~ carlo$ ls
```

```

drwx----- 3 paolo prof 102 May 18 22:49 Desktop
drwx----- 3 paolo prof 102 May 18 22:49 Documents
-rw-r--r-- 1 pippo stud 29 May 19 00:10 f1.txt
-rw-r--r-- 1 carlo prof 0 May 18 22:53 f2

```

numero di (hard) link

permessi

tipo di file

data ultima modifica

dimensione (byte)

nome file

proprietario gruppo

File Comandi in Shell 49

Protezione dei file

- Molti utenti

- ❑ Necessità di **regolare gli accessi** alle informazioni
- Per un file, esistono 3 tipi di utilizzatori:
 - ❑ proprietario, **user**
 - ❑ gruppo del proprietario, **group**
 - ❑ tutti gli altri utenti, **others**
- Per ogni tipo di utilizzatore, si distinguono tre modi di accesso al file:
 - ❑ **lettura (r)**
 - ❑ **scrittura (w)**
 - ❑ **esecuzione (x)** (per una directory significa list del contenuto)
- Ogni file è marcato con
 - ❑ **User-ID e Group-ID del proprietario**
 - ❑ **12 bit di protezione**

Bit di protezione

12 11 10 9 8 7 6 5 4 3 2 1

0	0	0	1	1	1	1	0	0	1	0	0
SUID	SGID	Sticky	R	W	X	R	W	X	R	W	X
			User			Group			Others		
			PERMESSI								

Sticky bit:

- **file eseguibili**: il sistema cerca di **mantenere in memoria l'immagine del programma**, anche se non è in esecuzione
- **directory**: file e directory all'interno di una directory con sticky bit a uno possono essere **eliminati solo dal root o dall'owner dei singoli file o directory**, anche se i permessi di singoli file o directory lo consentirebbero

SUID e SGID

- SUID (Set User ID) (identificatore di utente effettivo) □ Si applica a un file di **programma eseguibile solamente** □
Se vale 1, fa sì che **l'utente** che sta eseguendo quel programma **venga considerato il proprietario di quel file (solo per la durata della esecuzione)**
- È necessario per consentire operazioni di **lettura/scrittura su file di sistema**, che l'utente non avrebbe il diritto di leggere/modificare.
 - Esempio: **mkdir** crea un direttorio, ma per farlo deve anche **modificare alcune aree di sistema** (file di proprietà di root), che non potrebbero essere modificate da un utente. Solo SUID lo rende possibile
- SGID bit: come SUID bit, ma per il gruppo

Protezione e diritti su file

Per variare i bit di protezione:

❑ `chmod [u g o] [+ -] [rwx] <nomefile>`

I permessi possono essere concessi o negati dal solo **proprietario del file**

Esempi di variazione dei bit di protezione:

❑ `chmod 0755 /usr/dir/file`

0	0	0	1	1	1	1	0	1	1	0	1
SUID	SGID	Sticky	R	W	X	R	W	X	R	W	X
			User			Group			Others		

❑ `chmod u-w fileimportante`

Altri comandi:

❑ `chown <nomeutente> <nomefile>`

❑ `chgrp <nomegruppo> <nomefile>`

Comandi, piping e ridirezione

Comandi e input/output

I **comandi UNIX** si comportano come **FILTRI** □ filtro è un programma che riceve il suo **ingresso da standard input** e produce il **risultato su standard output** (trasformazione di dati)

Standard input

FILTRO

Default: tastiera
Standard
output

Standard
error

Default: monitor

Comandi shell Linux: filtri

Alcuni esempi:

❑ **grep** <testo> [<file>...]

Ricerca di testo. Input: (lista di) file. Output:

video ❑ **tee** <file>

Scriva l'input sia su file, sia sul canale di

output ❑ **sort** [<file>...]

Ordina alfabeticamente le linee. Input: (lista di) file.

Output: video

❑ **rev** <file>

Inverte l'ordine delle linee di file. Output:

video ❑ **cut** [-options] <file>

Seleziona colonne da file. Output: video

Ridirezione di input e output

Possibile ridirigere input e/o output di un comando facendo sì che non si legga da stdin (e/o non si scriva su stdout) **ma da file**

- ❑ **senza cambiare il comando**

- ❑ **completa omogeneità tra dispositivi e file**

- Ridirezione dell'input Aperto in lettura

- ❑ **comando <**

file_input • Ridirezione Aperto in scrittura
dell'output (nuovo o sovrascritto)

- ❑ **comando >**

file_output ❑ **comando** Scrittura in
>> file_output append

Esempi

- `ls -l > file`

File conterrà il risultato di `ls -l`

- `sort < file > file2`

Ordina il contenuto di `file` scrivendo il risultato su `file2`

- Cosa succede con `> file` ?

Piping

L'output di un comando può esser ridiretto a diventare l'input di un altro comando (piping)

- In DOS: **realizzazione con file temporanei** (primo comando scrive sul file temporaneo, secondo legge da questo)
- In UNIX: **pipe come costruito parallelo** (l'output del primo comando viene reso disponibile al secondo e consumato appena possibile, non ci sono file temporanei)
- Si realizza con il carattere speciale '|'

`comando1 | comando2`

`comando1 | comando2`

Esempi di piping

- `who | wc -l`
 - Conta gli utenti collegati
- `ls -l | grep ^d | rev | cut -d' ' -f1 | rev` □ Che cosa fa? Semplicemente mostra i nomi dei sottodirettori della directory corrente
 - `ls -l` lista i file del direttorio corrente
 - `grep` filtra le righe che cominciano con la lettera d (pattern `^d`, vedere il **man**)
 - ovvero le directory (il primo carattere rappresenta il tipo di file)
 - `rev` rovescia l'output di `grep`
 - `cut` taglia la prima colonna dell'output passato da `rev`, considerando lo spazio come delimitatore (vedi **man**)
 - quindi, poiché `rev` ha rovesciato righe prodotte da `ls -l`, estrae il nome dei direttori 'al contrario'
 - `rev` raddrizza i nomi dei direttori

Suggerimento: aggiungere i comandi uno alla volta (per vedere cosa viene prodotto in output da ogni pezzo della pipe)

Esecuzione di comandi in Shell

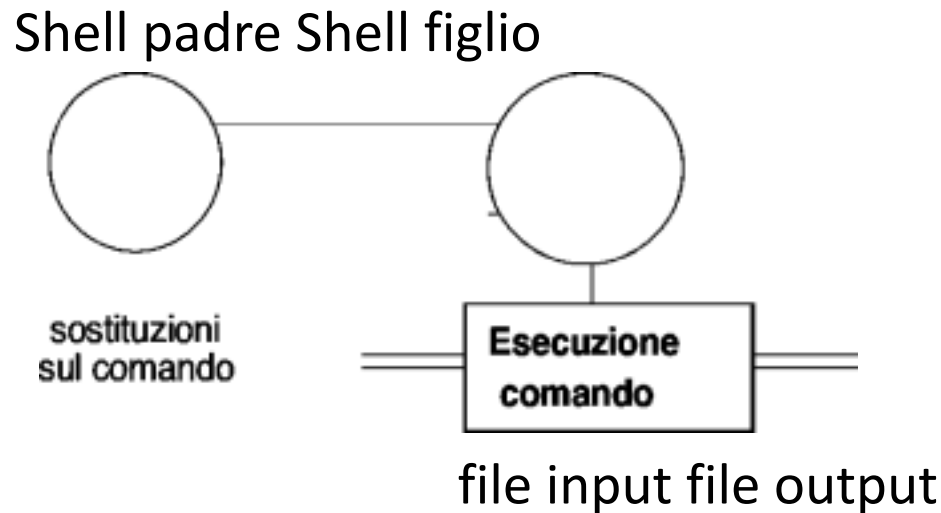
- In UNIX alcuni comandi sono eseguiti direttamente dalla shell corrente (comandi built-in)
- Invece **molti comandi sono eseguiti da una nuova shell** creata al momento
- La shell attiva mette in esecuzione una seconda shell che:
 - ❑ esegue le sostituzioni dei metacaratteri e dei parametri (parsing)
 - ❑ cerca il comando
 - ❑ esegue il comando



Shell padre Shell figlio

- La shell padre attende il completamento dell'esecuzione della sotto-shell (comportamento sincrono)

Esecuzione di un comando



Il comando esegue avendo collegato

- il proprio input al file di input
 - il proprio output al file di output
- specificati dalla shell di lancio

Schema di un processore di comandi

procedure shell (ambiente, filecomandi);

< eredita ambiente (esportato) del padre, via copia>

begin

repeat

<leggi comando da filecomandi>

if < **è comando built-in** > then

<modifica direttamente ambiente>;

else if <**è comando eseguibile**> then

<esecuzione del comando via nuova shell>

else if <**è nuovofilecomandi**> then

shell (ambiente, nuovofilecomandi);

else < errore>;

CTRL-C

endif

until <fine file>

end shell;

Per abortire il comando corrente:

Non creare un file comandi
ricorsivo senza una
condizione di terminazione!!!

Rientranza della shell

- Una shell è un programma che esegue i comandi, forniti da terminale o da file
- Si invocano le shell come i normali comandi eseguibili con il loro nome

sh [<filecomandi>]

csh [<filecomandi>]

- **Le invocazioni attivano un processo che esegue la shell**
- Le shell sono **RIENTRANTI**: più processi possono condividere il codice senza errori e interferenze

sh

csh

ps # quanti processi si vedono?

Metacaratteri ed espansione

Metacaratteri

Shell riconosce **caratteri speciali (wildcard)**

***** una qualunque stringa di zero o più caratteri in un nome di file ?

[] un qualunque carattere in un nome di file

[zfc] un qualunque carattere, in un nome di file, compreso tra quelli nell'insieme. Anche **range** di valori: **[a-d]**. Per esempio **ls [q-s]*** lista i file con nomi che iniziano con un carattere compreso tra q e s

commento fino alla fine della linea

**** escape (segnala di **non interpretare** il carattere)

Esempi con metacaratteri

```
ls [a-p,1-7]*[c,f,d]?
```

- elenca i file i cui nomi hanno come iniziale un carattere compreso tra 'a e 'p' oppure tra 1 e 7, e il cui penultimo carattere sia 'c', 'f', o 'd'

```
ls *\**
```

- Elenca i file che contengono, in qualunque posizione, il carattere *

Variabili nella shell

In ogni shell è possibile **definire un insieme di variabili** (trattate come stringhe) con **nome e valore**

- ❑ i riferimenti ai **valori delle variabili** si fanno con il **carattere speciale \$** (`$nomevariabile`)

- ❑ si possono fare **assegnamenti**

`nomevariabile=$nomevariabile`

l-value r-value

Esempi

- ❑ `x=2` # !!! NO spazio prima/dopo =

- ❑ `echo $X` # visualizza 2
- ❑ `echo $PATH` # mostra il contenuto della variabile PATH
- ❑ `PATH=/usr/local/bin:$PATH` # aggiunge la directory /usr/local/bin alle directory del path di default

File Comandi in Shell 68

Ambiente di esecuzione

Ogni comando esegue **nell'ambiente associato (insieme di variabili di ambiente definite) alla shell** che esegue il comando

- ogni shell **eredita l'ambiente dalla shell** che l'ha creata
- nell'ambiente ci sono **variabili** alle quali il comando può fare riferimento:

- ❑ **variabili con significato standard: PATH, USER, TERM, ...**

❑ variabili user-defined

File Comandi in Shell 69

Variabili

Per vedere tutte le variabili di ambiente e i valori loro associati si può utilizzare il comando **set**:

```
BASH=/usr/bin/bash
```

```
HOME=/space/home/wwwlia/www
```

```
PATH=/usr/local/bin:/usr/bin:/bi
```

```
n PPID=7497
```

```
PWD=/home/Staff/CarloGiannelli
```

```
i SHELL=/usr/bin/bash
```

```
TERM=xterm
```

```
UID=1015
```

```
USER=cgiannelli
```

File Comandi in Shell 70

Espressioni

Le **variabili shell** sono **stringhe**. È comunque possibile **forzare l'interpretazione numerica** di stringhe che contengono la codifica di valori numerici

❑ comando **expr**:

```
expr 1 + 3
```

Esempio:

```
var=4
```

```
echo risultato: $var + 1
```

`$var + 1` è il risultato della corrispondente espressione? a
che cosa

```
echo risultato:
```

serve?

```
`expr $var + 1`
```

File Comandi in Shell 71

Esempio

```
#!/bin/bash file somma A=5
```

```
B=8
```

```
echo A=$A, B=$B
```

```
C=`expr $A + $B`
```

```
echo C=$C
```

```
bash$ somma
```

```
A=5, B=8
```

```
C=13
```

output

File Comandi in Shell 72

Espressioni, notazione alternativa

Per operazioni tra interi invece di **expr(...)** è possibile utilizzare:

- **\$((...))** con restituzione del risultato
- **((...))** senza restituzione del risultato

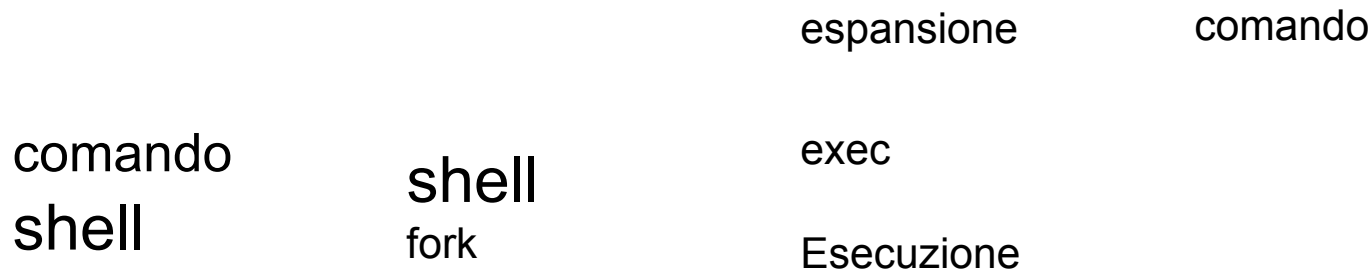
Ad esempio:

- `expr $a + $b → $ ( ($a+$b) )`

- `c=expr $a + $b → c=$(( $a+$b ))`
oppure `((c=$a+$b))`

File Comandi in Shell 73

Espansione/Parsing: sostituzione della shell



Prima della esecuzione, il comando viene scandito (**parsing**),

alla ricerca di caratteri speciali (*, ?, \$, >, <, |, etc.) • La shell **prima prepara i comandi come filtri**: ridirezione e piping di ingresso uscita

- Nelle successive scansioni, se shell trova altri caratteri speciali, **produce delle sostituzioni (passo di espansione)**

File Comandi in Shell 74

Passi di sostituzione

Sequenza dei passi di sostituzione

1) Sostituzione dei comandi

- comandi contenuti tra ``...`` (**backquote**) o tra `$(...)` sono eseguiti e sostituiti dal risultato prodotto **2)**

Sostituzione delle variabili e dei parametri • **nomi delle variabili** (`$nome`) sono espansi nei valori

corrispondenti

3) Sostituzione dei metacaratteri in nomi di file •

metacaratteri * ? [] sono espansi nei **nomi di file** secondo un meccanismo di **pattern matching**

File Comandi in Shell 75

Inibizione dell'espansione

In alcuni casi è necessario **privare i caratteri speciali del loro significato**, considerandoli come caratteri normali □ \
carattere successivo è considerato come un normale
carattere

□ ' ' (apici): proteggono da qualsiasi tipo di
espansione

→ NO 1, 2 e 3 slide precedente

□ " " (doppi apici) proteggono dalle espansioni con

l'eccezione di \$ \ ` ` (backquote)

→ NO 3 slide precedente, Sl' 1 e 2

File Comandi in Shell 76

Esempi inibizione dell'espansione

y=3

echo ' * e \$y ' # produce * e \$y

echo " * e \$y " # produce * e 3

echo "`pwd`" # stampa nome dir corrente echo

"\$(pwd)" # stampa nome dir corrente

sia " che ' impediscono alla shell di interpretare i caratteri speciali per la ridirezione (<, >, >>) e per il piping (|)

Comando eval

```
y=3
```

```
x='$y'
```

```
echo $x # stampa $y (perché x vale $y)
```

Per ottenere una ulteriore sostituzione, se richiesta, bisogna forzarla → comando **eval**

```
eval echo $x # stampa 3 (perché valuta $y)
```

eval esegue il comando passato come argomento (dopo che la shell ne ha fatto il parsing).

eval consente quindi una ulteriore fase di sostituzione.

Esempio eval

Esempio:

```
$ p='ls|more'
```

```
$ $p
```

```
ls|more: command not found
```

```
$ eval $p
```

```
Ascii_pic
```

```
Bfile
```

```
...
```

Altri esempi

- `rm '$var'*`
 - ❑ Rimuove i file che cominciano con `*$var`
- `rm "$var"*`
 - ❑ Rimuove i file che cominciano con `*<contenuto della variabile var>`
- `host203-31:~ carlo$ echo "<`pwd`>"`
`</Users/carlo>`
- `host203-31:~ carlo$ echo`
`'<`pwd`>' <`pwd`>`
- `A=1+2 B=`expr 1 + 2``
 - ❑ In A viene memorizzata la stringa `1+2`, in B la stringa `3`

(expr forza la valutazione aritmetica della stringa passata come argomento, i backquote forzano la valutazione di expr)